



نظري

1



10



1200 sp

كلية الهندسة المعلوماتية

السنة الثانية

البرمجة بلغة Assembly

د. عبير ميا



RB Informatics; 22/04/2025

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

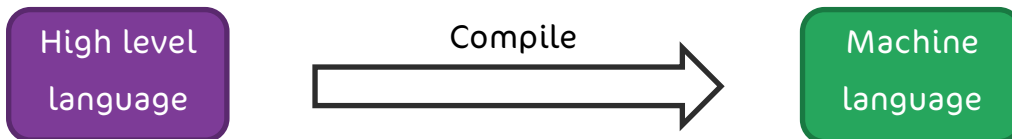
■ محاور محاضرة اليوم:

- أهمية لغة Assembly
- السجلات Registers
- معالج MIPS
- أنماط التعليمات في معالج MIPS

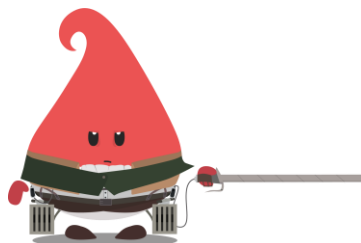
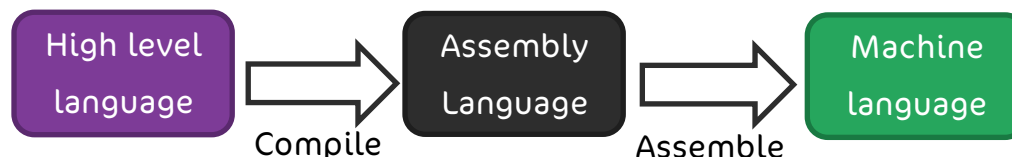
مقدمة

اللغات عالية المستوى مثل C++, Java, JavaScript... تتميز بكونها لغات سهلة الفهم بالنسبة للمبرمج لكنها غير مفهومة بالنسبة للمعالج الذي يتعامل فقط بلغة الآلة Machine language. هنا، يأتي دور المترجم Compiler الذي يحول اللغات عالية المستوى إلى لغة الآلة. للCompilers أنواع:

1. High-level language to machine language compilers.



2. High-level language to Assembly language to Machine language.

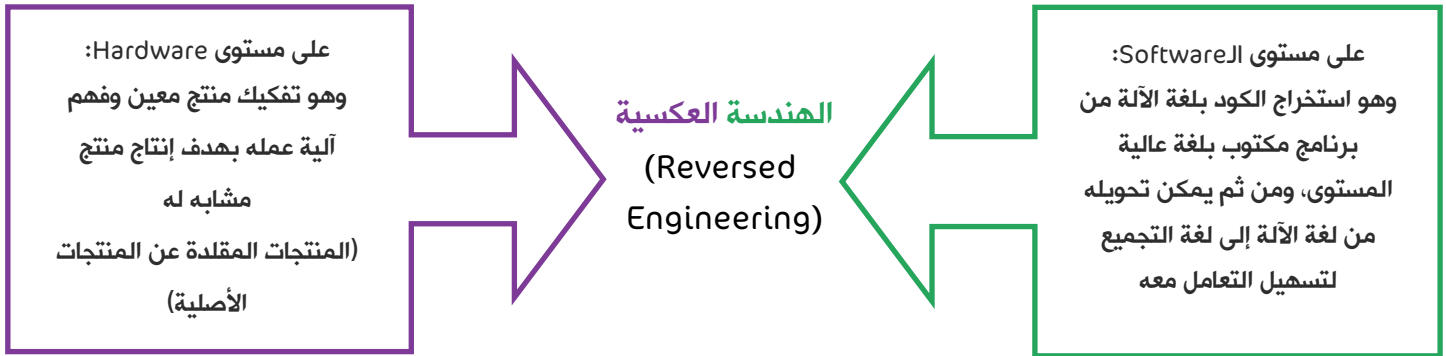


لماذا تتعلم لغة الـ Assembly؟

- تتيح للمبرمج فهم كيفية عمل المعالج وإدارته لموارد الحاسوب مما يطور مهارات المبرمج باستخدام الـ High-level languages ويؤدي إلى كتابة برامج ذات كفاءة أكبر
- هذه اللغة تتواصل مباشرة مع المعالج (دون الحاجة إلى Compiler).
 ⇐ كتابة البرامج باستخدامها يتيح تحكم أكبر بالموارد، ويكون وقت تنفيذها سريعاً.
- تستخدم لبرمجة الأنظمة المدمجة (Embedded systems)
 ⇐ مثال البرادات الحديثة والغسالات والسيارات الحديثة.
- يمكن استخدام لغة التجميع Assembly language لبرمجة أجزاء من الكود يصعب على اللغات عالية المستوى تنفيذها.



- تستخدم بالهندسة العكسية (Reversed Engineering) والتي تكون على شكلين:



ومن مرحلة الـ Software reversed engineering يمكن القيام بالعديد من الأشياء، مثلاً:

- كسر حماية برنامج (Cracking): مثلاً بفرض كان البرنامج محمي بـ Serial number، فيمكننا من خلال كود الـ Assembly الوصول إلى الجزء الخاص بإدخال هذا الـ Serial number، والتعديل عليه.
- بفرض كان هذا البرنامج يعمل لفترة محدودة فقط (Trial version) (أو البرامج المدفوعة التي تقدم شهر واحد مجاني كتجربة)، عندها يمكن من خلال الوصول إلى كود الـ Assembly الخاص بالبرنامج الوصول إلى الجزئية الخاصة بتحديد المدة التجريبية وتمديدتها.
- يمكن التعديل على برنامج ذو إصدار لا يعمل على حاسبنا وتعديل بعض خواصه ليعمل.
- بفرض تعرض الحاسب لتدخل فيروسي فيمكن الحصول على كود الـ Assembly الخاص بالفيروس (لأن الفيروس عبارة عن برنامج) ومن خلاله يمكن إكتشاف الثغرات الموجودة فيه. من أجل تطوير برنامج لمكافحة هذا النوع من الفيروسات.
- بفرض أصدرت إحدى الشركات المنافسة برنامج وكان به ثغرة أمنية معينة، فيمكن من خلال الحصول على كود الـ Assembly الخاص بالبرنامج معرفة الثغرة الأمنية لتجنبها عند صناعة البرنامج الخاص بشركتنا.

البرمجة بلغة التجميع Assembly

يتم التعامل مع السجلات Registers بشكل مباشر بدلاً من المتحولات Variables، لذا يجب معرفة عدد السجلات الموجودة في المعالج الذي نتعامل معه وادارتها بالشكل الأمثل حتى تخدم البرنامج الذي نكتبه.

❖ **ملاحظة: كل Compiler يعمل بشكل مختلف عن الـ Compilers الأخرى، فهي تختلف بالأداء وزمن التنفيذ.**

في معالجات 8086:

التعليمة `add a1,b1` تقوم بجمع محتويات السجلات `a1` و `b1` معاً وتضع الناتج في `a1`.

في معالجات MIPS:

التعليمة `add $10, $9, $8` تقوم بجمع محتوى السجل **التاسع** مع محتوى السجل **الثامن** وتضع الناتج في السجل **العاشر**.

1. المعالج الذي سنستخدمه في درساتنا هو معالج MIPS R3000 والمؤلف من 32 سجل وكل منها مؤلف من 32 bit (كل قلاب Flip-Flop يحفظ bit واحد)
2. يمكن الإشارة إلى أي سجل إما برقمه أو بإسمه، مسبقاً بإشارة \$ مثل (\$t1, \$0, \$9...)
3. السجلات مرقمة من \$0 إلى \$31
4. محتوى السجل الصفري \$0 دائماً يساوي الصفر ولا يمكن تعديله.
5. يمكن إضافة comments بوضع إشارة # قبل الكتابة

❖ **جدول يوضح أسماء السجلات وأرقامهن (غير مطالبين بحفظهم)**

#	Name
0	Zero
2	at
2, 3	v0, v1
4,...,7	a0, a3
8,...,15	t0,...,t7
16,...,23	s0,...,s7
24, 25	t8,t9
26, 27	
28	
29, 30	
31	

تذكرة:

Machine language:

هي لغة الأصفار والواحدات 0s and 1s والتي يتعامل معها المعالج.

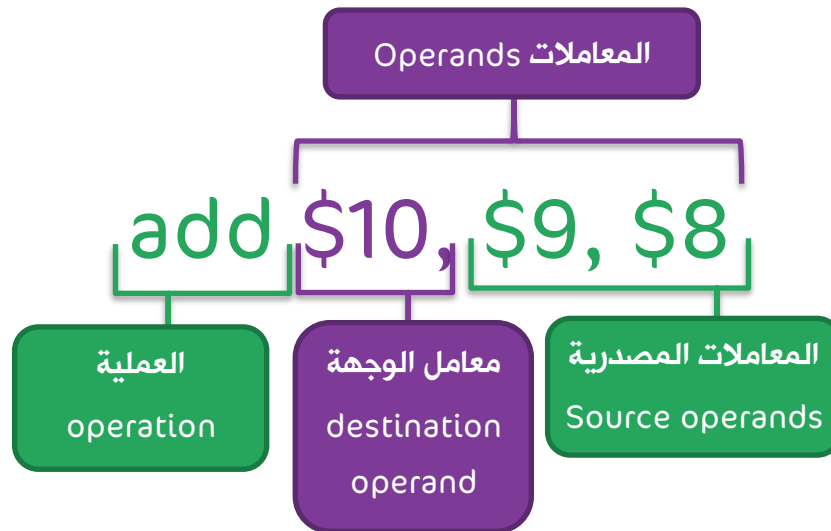
Assembly language:

لغة وسيطة بين اللغات عالية المستوى ولغة الآلة

ملاحظة:

للسجلات \$0 و \$1 ومن \$27 إلى \$31 استخدامات خاصة سنتعلمها قريباً. يفضل عند البرمجة استخدام السجلات من \$2 إلى \$26.

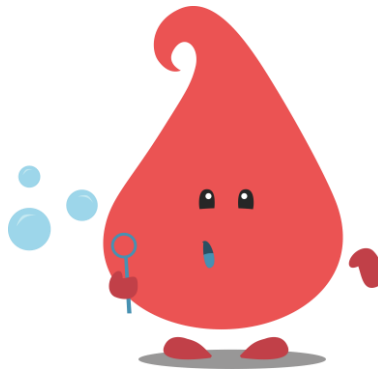
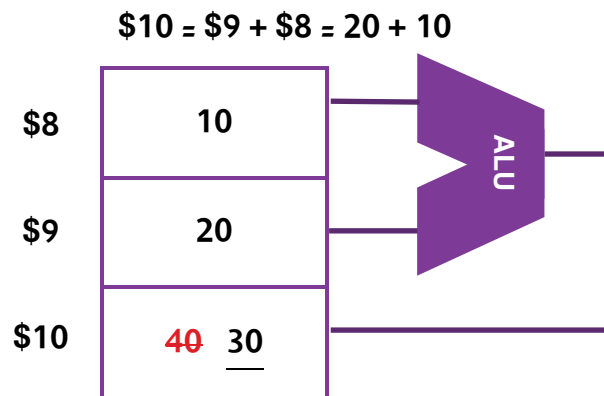
مثال: تعليمة الجمع محتوى سجلين ووضع الناتج في سجل ثالث



وبفرض كانت القيم الابتدائية للسجلات كالتالي:

$$\begin{array}{lll} \$10 = 0 & \$9 = 20 & \$8 = 10 \end{array}$$

سيتم الآتي:



مثال : تعليمة جمع محتوى سجل مع قيمة عددية ووضع الناتج في سجل.

addi \$10, \$9, 5

معامل مباشر
Immediate Operand

يمكن للمعامل المباشر أن يكون قيمة عددية إما بالنظام العشري أو الست عشري أو الثنائي سواء سالب أو موجب.
ملاحظة: يجب حفظ ترتيب المعادلات وعددها لكل تعليمة، فهي لا تتغير.

أمثلة:

■ كيف يمكن شحن السجل \$8 بالقيمة 27 باستخدام تعليمة واحدة؟

addi \$8, \$0, 27

ويمكن تمثيل القيمة العددية بالنظام الست عشري أي يمكن كتابتها كالاتي:

addi \$8, \$0, 0x1B

■ ما محتوى السجلين \$9, \$8 بعد تنفيذ العمليتين الآتيتين؟

addi \$9, \$0, 21

addi \$8, \$0, 0x21

السجل \$8:

$$\$8 = 0 + 0x21 = (0x21)_{16} = (33)_{10}$$

السجل \$9:

$$\$9 = 0 + 21 = (21)_{10}$$

■ ملاحظة:

توجد التعليمات add, or, and... ويقابل كل منها التعليمات addi, or, andi... لكن لا يوجد لتعليمة الطرح sub مقابل subi وذلك لأنه يمكن الاستعاضة عنها باستخدام addi حيث يكون المعامل المباشر immediate operand ذو قيمة سالبة. والسبب في عدم وجود تعليمة subi هو أن معالج MIPS R3000 من نوع RISC (Reduced Instruction Set Computer)، أي أنه يحتوي على عدد محدود من التعليمات.
لكن:

عند كتابة التعليمة subi \$10, \$9, 2 مثلاً على البرنامج وتنفيذها سيعطي خطأً صحيحاً. وهذا ما يدعى بالتعليمات الزائفة، وهي التعليمات التي تنفذ بشكل صحيح على البرنامج لكن ليس لها تصميم فعلي على المعالج.

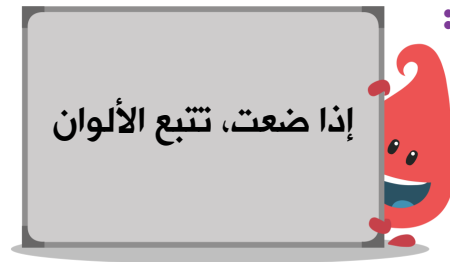
Binary	Decimal	Hexadecimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

2^1	2	$(10)_2$
2^2	4	$(100)_2$
2^3	8	$(1000)_2$
2^4	16	$(10000)_2$
2^5	32	$(100000)_2$
2^6	64	$(1000000)_2$
2^7	128	$(10000000)_2$
2^8	256	$(100000000)_2$
2^9	512	$(1000000000)_2$
2^{10}	1024	$(10000000000)_2$

ملاحظة: نستخدم النظام الست عشري عند التعامل مع عناوين الذاكرة.

أمثلة:

- $(4)_{10} = (2^2)_{10} = (100)_2$
- $(64)_{10} = (2^6)_{10} = (1000000)_2$
- $(1024)_{10} = (2^{10})_{10} = (10000000000)_2$
- $(24)_{10} = (12 \times 2)_{10} = (11000)_2$



في النظام الثنائي، عند الضرب بـ 2 نضيف 0 على يمين الرقم (مثل الضرب بـ 10 في النظام العشري) وعند الضرب بـ 4 نضيف صفراً (مثل الضرب بـ 100 في النظام العشري) وهكذا...

- $(44)_{10} = (11 \times 4)_{10} = (101100)_2$
- $(127)_{10} = (128 - 1)_{10} = (10000000 - 1)_2 = (1111111)_2$
- $(513)_{10} = (512 + 1)_{10} = (1000000000 + 1)_2 = (1000000001)_2$
 $(0x2C)_{16} + (0x4)_{16}$
 $(C)_{16} + (4)_{16} = (12)_{10} + (4)_{10} = (16)_{10} = (0x10)_{16}$
 $(0x10)_{16} + (0x20)_{16} = (0x30)_{16}$
- $(30)_{10} = (15 \times 2)_{10} = ((16 - 1) \times 2)_{10} = ((10000 - 1) \times 10)_2 = (11110)_2$
- $(40)_{10} = (10 \times 4)_{10} = (101000)_2$
- $(63)_{10} = (64 - 1)_{10} = (2^6 - 1)_{10} = (1000000 - 1)_2 = (111111)_2$
- $(0x1F)_{16} + (0x1)_{16}$
 $(F)_{16} + (1)_{16} = (16)_{10} + (1)_{10} = (17)_{10} = (0x10)_{16}$
 $(0x10)_{16} + (0x1)_{16} = (0x11)_{16}$
- $(0x1008)_{16} + (0x4)_{16} = (0x100C)_{16}$

أنماط التعليمات في MIPS

(MIPS instruction formats)

كل تعليمة ترمز بـ 32 بت، لها 3 أنواع:

R-Type

add, sub, and, or, ...

ويكون شكل التعليمة في هذا النمط بالشكل التالي:

OP	rs	rt	rd	shift	func
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- ترميز الـ 6 بت الأولى (الخاصة بالـ OP) تكون 000000 في هذا النمط (أي أن $OP = 0$).
- الـ R ترمز إلى Register أو سجل، حيث أن هذه التعليمات تتعامل مع السجلات فقط (جميع معاملاتها سجلات).
- عملية الجمع تكون من الشكل التالي: add rd, rs, rt
- ملاحظة:** نلاحظ اختلاف ترتيب المعاملات rd, rs, rt عند كتابة التعليمة بالـ bits عن الـ Assembly.

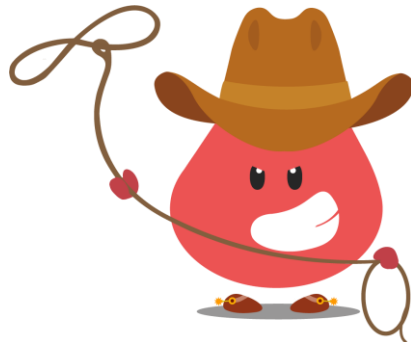
I-Type

addi, ori, andi, lw, sw, beq, ...

ويكون شكل التعليمة في هذا النمط بالشكل التالي:

OP	rs	rt	immediate
6 bits	5 bits	5 bits	16 bits

- ترميز الـ 6 بت الأولى (الخاصة بالـ OP) تحقق أن $(OP \neq 0, 2, 3)$
- ترمز الـ Immediate، حيث أن هذه التعليمات تتعامل مع المعاملات المباشرة.
- عملية الجمع تكون من الشكل التالي: addi rt, rs, imm



J-Type

JLabel (OP = 2)

Jal func (OP = 3)

ويكون شكل التعليم في هذا النمط بالشكل التالي:

OP	Immediate (Target address)
6 bits	26 bits

6. **ملاحظة:** عند كتابة التعليم على شكل سلسلة مؤلفة من 23 بت من الأصفار والواحدات يجب أن:
7. ننظر إلى أول 6 بت والتي تمثل الـ OP، حتى نحدد نمط التعليم.
8. إذا كانت $OP = 0$ (000000) $\Leftarrow$ التعليم R-Type عندها ننظر إلى آخر 6 بت والتي تمثل الـ func والتي تحدد التعليم إذا كانت add أو or أو...

في الامتحان يتم إرفاق جدول يعبر عن كل تعليمة والـ func المقابلة لها.

إذا كان ترميز الـ OP هو 000010 فالتعليمة J-Type وهي JLabel

إذا كان ترميز الـ OP هو 000011 فالتعليمة J-Type وهي Jal func

إذا كان ترميز الـ OP هو 000000 فالتعليمة R-Type فننظر للـ func لمعرفة التعليمة بالتحديد

Example: 000011 01100101110010101101...

نلاحظ أن ترميز الـ OP هو 3 ومنه فإن التعليمة هي J-Type وهي JLabel .

Example: 000000 10000 00000 01100 00000 010000

$OP = 0$ ومنه فإن التعليمة R-Type $\Leftarrow$ ننظر إلى ترميز الـ func فنجد أن قيمته تساوي 16 في النظام العشري ومنه فإن التعليمة هي or

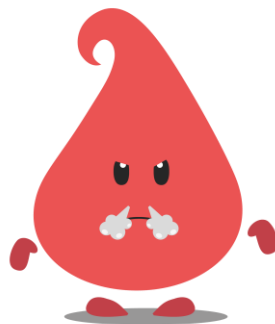
$$rs = (10000)_2 = (2^4)_{10} = (16)_{10}$$

$$rt = (00000)_2 = (0)_{10}$$

$$rd = (01100)_2 = (12)_{10}$$

ومنه فإن التعليمة كاملة في لغة التجميع هي:

Or \$12, \$16, \$0

تنسأاش ترتيب المعاملات في هذا النوع من التعليمات rd, rs, rt

٤ أي جزء من المعالج يحل التعليمات السابقة؟

الإجابة: تقوم وحدة التحكم الـ (CU) بالعملية السابقة للتعرف على التعليمات.

مثال: نفرض القيم الابتدائية للسجلات هي:

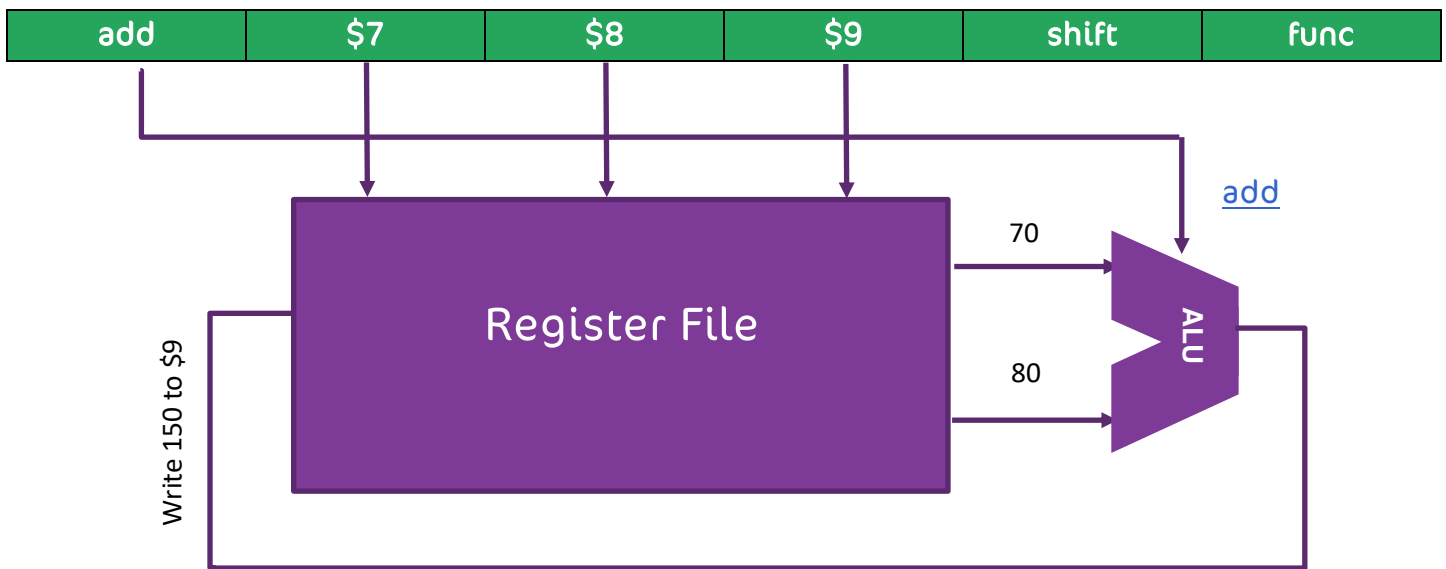
$$\$7 = 70 \quad \$8 = 80 \quad \$9 = 90$$

ولدينا التعليمات التالية:

`add $9, $7, $8`

والناتج هو 150 وسيخزن في السجل \$9.

فيتم تمثيل التعليمة كالتالي:

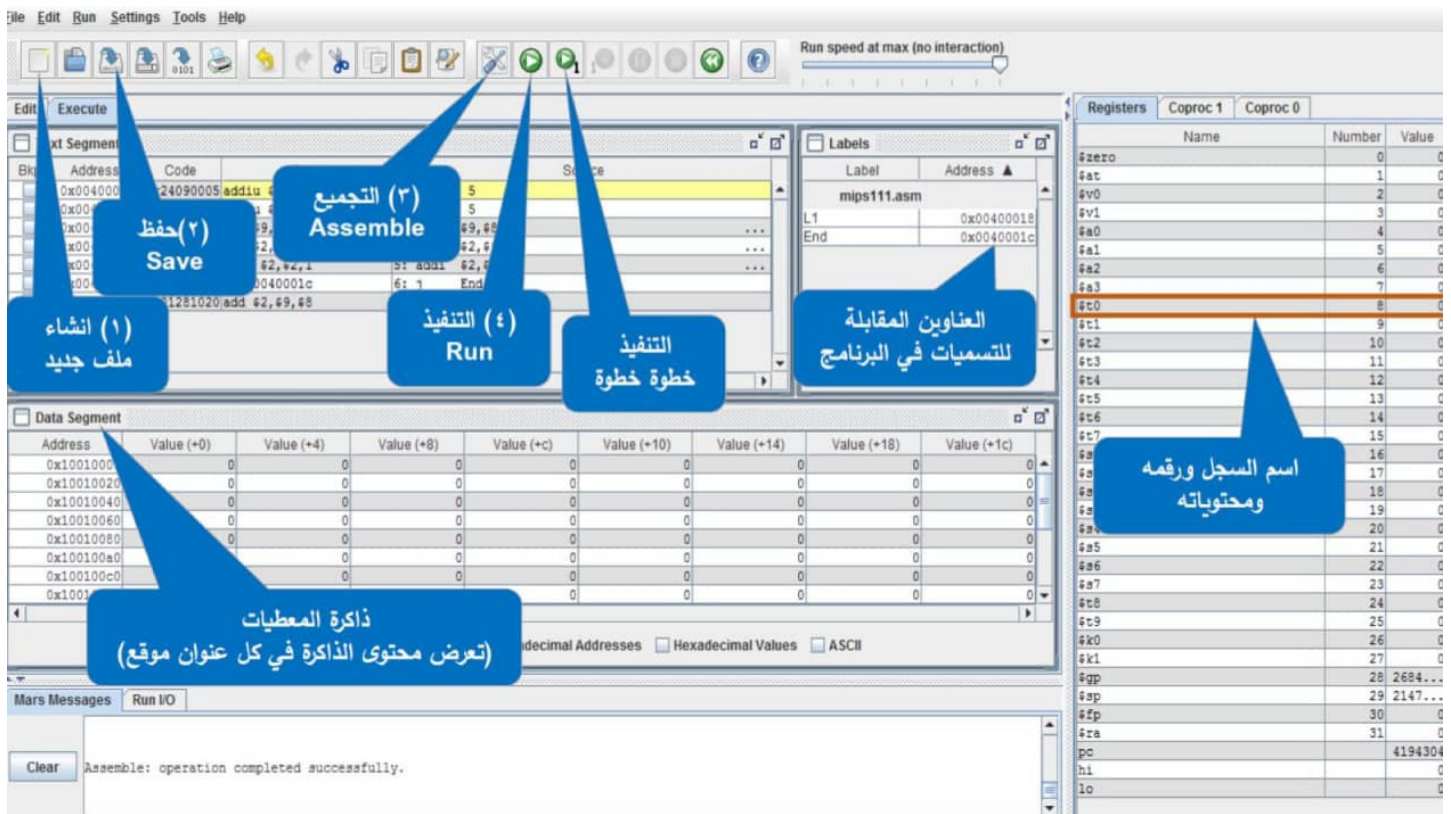


مثال: بفرض أن المحتويات الابتدائية لكل سجل هي: (رقم السجل + 10) مثلا ($\$3 = 3 + 10 = 13$). وبفرض لدينا التعليمات المتسلسلة الآتية. عبر عن محتوى كل سجل rd بعد كل عملة تنفيذ.

1. `sub $8, $1, $8` $\#\$8 = 11 - 18 = -7$
2. `addi $9, $9, -8` $\#\$9 = 19 - 8 = 11$
3. `andi $2, $5, 0xB` $\#\$2 = (1111)_2 \text{ and } (1011)_2 = (1011)_2 = (11)_{10}$
4. `or $11, $7, $4` $\#\$11 = 17 \text{ or } 14 = (10001)_2 \text{ or } (01110)_2 = (11111)_2 = (31)_{10}$
5. `and $12, $0, $5` $\#\$12 = 0 \text{ and } 15 = (0000)_2 \text{ and } (1111)_2 = (0)_{10}$
6. `or $10, $0, $9` $\#\$10 = 0 \text{ or } 9 = (0000)_2 \text{ or } (1001)_2 = (1001)_2 = (9)_{10}$
7. `sub $13, $8, $10` $\#\$13 = -7 - 20 = -27$

برنامج MARS

- هو برنامج محاكاة لتنفيذ البرامج المكتوبة بلغة التجميع Assembly من اجل معالجات MIPS، وهو اختصار للعبارة MIPS Assembler and Runtime Simulator، فهو إذاً مجمع وبنفس الوقت برنامج محاكاة لمعالج MIPS أثناء التنفيذ.
- دخل البرنامج بلغة التجميع وخرجه هو نتيجة تنفيذ البرنامج.
- يمكن تحميل برنامج المحاكاة MARS من الرابط:
- <https://www.cs.missouristate.edu/MARS>
- البرنامج مكتوب بلغة جافا ويلزم توفر الإصدار 1.5 أو ما بعد Java SE Development Kit على الجهاز حتى يعمل البرنامج ويتم توزيعه على شكل ملف JAR قابل للتنفيذ.
- بالنقر المزدوجة على الأيقونة Mars.jar تظهر واجهة البرنامج الرسومية كما في الشكل التالي:



The END....